

Fit advanced SECR models

Ian Durbach & Cornelia Oedekoven

Covered in this tutorial

- Fit full-survey advanced models
- Fit shortened-survey models using the same fitting mask
- Fit multisession models with session-specific masks and `userdist`
- Fit sex-specific full-survey models using `hcov = "sex"` and `groups = "sex"`
- Compare each candidate set with AIC
- Save fitted models for the results tutorial

Introduction

This tutorial fits four related advanced model sets. The full-survey and shortened-survey analyses are closest to the basic model-fitting workflow, but they use the elevation-constrained fitting mask and gorge-avoiding movement distances. The multisession analysis uses a list of capture histories, masks, and `userdist` objects. The sex-specific analysis demonstrates two ways to include individual sex information.

The model sets are deliberately small and explicit. This keeps the teaching focus on how the advanced inputs change the fitting call, rather than on a large model-selection exercise.

Preparations

Only `secr` is needed for fitting because the capture histories, masks, and user-distance objects have already been created.

```
# Packages needed  
library(secr)
```

Encounter and detection functions

The advanced models use the hazard half-normal detection function, `detectfn = "HHN"`. With count detectors, it is often more useful to think about the encounter-rate function, $\lambda(d)$, because an individual can be detected more than once during an occasion. Detection probability, $g(d)$, is then derived from the encounter rate.

For the hazard half-normal model, encounter rate declines with distance from an activity centre to a camera. `lambda0` is the encounter-rate parameter at distance zero, and `sigma` controls the spatial scale over which encounter rate declines. Both are fitted on a link-function scale, so we usually interpret predictions on the natural scale rather than reading raw coefficients directly.

Trap covariates such as `cliff` and `valley_stream` enter the `lambda0` model. Mask covariates such as `std_tri` and `std_elev` enter the density model, `D`. Keeping those roles distinct is especially important in the advanced tutorial because the mask and detector covariates come from different spatial objects.

Full-survey models

The full-survey model set is the closest advanced equivalent to the basic model-fitting tutorial. The two important changes are that the fitting mask is `mask_model`, which has been clipped to plausible elevations, and that `details = list(userdist = userd_model)` tells `secr.fit()` to use gorge-avoiding movement distances.

```
output_file <- "tutorials/namkha_advanced/output/namkha_advanced_fitted_models_full.RData"

load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_capthist.RData")
load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_mask.RData")

# Check the capture history and covariates before fitting models.
summary(full_ch)
```

Object class	capthist
Detector type	count
Detector number	55
Average spacing	2504.368 m
x-range	534489 576620 m
y-range	3316462 3360989 m

Usage range by occasion

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
min	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
max	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	5

Counts by occasion

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
n	0	0	2	3	4	2	3	7	2	4	4	2	2	3	4	2	0	2	0	0
u	0	0	2	3	4	1	2	5	2	2	2	0	2	1	0	0	0	0	0	0
f	12	10	2	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
M(t+1)	0	0	2	5	9	10	12	17	19	21	23	23	25	26	26	26	26	26	26	26
losses	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
detections	0	0	2	3	4	2	3	7	3	4	4	2	2	3	4	2	0	2	0	0
detectors visited	0	0	2	3	4	2	2	7	3	4	4	2	2	3	4	2	0	2	0	0
detectors used	3	6	18	16	21	33	35	32	30	29	28	28	20	18	12	7	11	15	9	8

Total

n	46
u	26
f	26
M(t+1)	26
losses	0
detections	47
detectors visited	46
detectors used	379

Individual covariates

sex

Female : 8
Male : 15
Unknown: 3

```
names(covariates(full_traps))
```

```
[1] "Topography" "cliff"      "hill"      "valley_stream"
[5] "Water"      "Altitude"
```

```
names(covariates(mask_model))
```

```
[1] "tri"      "d2hydro"  "elev"     "std_tri"  "std_d2hydro"
[6] "std_elev"
```

The fitted models use the hazard half-normal detection function. D is modelled with mask covariates such as ruggedness and elevation; λ_0 is modelled with detector covariates such as cliff and valley/stream.

```
f0 <- secr.fit(
  full_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
  trace = FALSE
)

f1 <- secr.fit(
  full_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ 1, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
  start = f0,
  trace = FALSE
)

f2 <- secr.fit(
  full_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_elev, lambda0 ~ 1, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
  start = f0,
  trace = FALSE
)

f3 <- secr.fit(
  full_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ 1, lambda0 ~ valley_stream, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
```

```

    start = f0,
    trace = FALSE
  )

f4 <- secr.fit(
  full_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ cliff + valley_stream, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
  start = f1,
  trace = FALSE
)

AIC(f0, f1, f2, f3, f4)

```

	model	detectfn	npar
f4	D~std_tri lambda0~cliff + valley_stream sigma~1	hazard halfnormal	6
f3	D~1 lambda0~valley_stream sigma~1	hazard halfnormal	4
f0	D~1 lambda0~1 sigma~1	hazard halfnormal	3
f2	D~std_elev lambda0~1 sigma~1	hazard halfnormal	4
f1	D~std_tri lambda0~1 sigma~1	hazard halfnormal	4

	logLik	AIC	AICc	dAIC	AICwt
f4	-104.9857	221.971	226.392	0.000	0.6045
f3	-107.5876	223.175	225.080	1.204	0.3311
f0	-110.8342	227.668	228.759	5.697	0.0350
f2	-110.6667	229.333	231.238	7.362	0.0152
f1	-110.7390	229.478	231.383	7.507	0.0142

When fitting many models, it is good practice to check whether the optimiser finished cleanly and whether any coefficient estimates are NA or infinite. `trace = FALSE` suppresses the iteration output from `secr.fit()`, but it does not remove the need to inspect warnings and fitted objects.

```

full_fit_checks <- data.frame(
  model = c("f0", "f1", "f2", "f3", "f4"),
  fit_code = sapply(list(f0, f1, f2, f3, f4), function(x) x$fit$code),
  any_na_coef = sapply(list(f0, f1, f2, f3, f4), function(x) any(is.na(coefficients(x)))),
  any_inf_coef = sapply(list(f0, f1, f2, f3, f4), function(x) any(is.infinite(abs(unlist(coef
)
full_fit_checks

```

	model	fit_code	any_na_coef	any_inf_coef
1	f0	1	FALSE	FALSE
2	f1	2	FALSE	FALSE
3	f2	1	FALSE	FALSE
4	f3	1	FALSE	FALSE
5	f4	1	FALSE	FALSE

The fitted coefficients returned by `coef()` are on the link scale. For positive parameters such as `lambda0` and `sigma`, the natural-scale estimates are obtained by applying the inverse link function. `predict()` does this transformation for the fitted model and is usually the safer way to inspect interpretable parameter estimates.

```
coef(f0)
```

	beta	SE.beta	lcl	ucl
D	-8.773301	0.2536471	-9.270440	-8.276162
lambda0	-6.171899	0.2815289	-6.723686	-5.620113
sigma	8.938418	0.1476587	8.649013	9.227824

```
predict(f0)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.548117e-04	3.990767e-05	9.416705e-05	2.545121e-04
lambda0	log	2.087268e-03	5.994643e-04	1.202099e-03	3.624232e-03
sigma	log	7.619137e+03	1.131192e+03	5.704512e+03	1.017638e+04

The fitted models are saved separately from the shortened, multisession, and sex-specific model sets.

```
save(
  f0, f1, f2, f3, f4,
  file = output_file
)
```

Shortened-survey models

The shortened-survey analysis is one response to closure concerns. It uses `short_ch` and `short_traps`, which already contain detections and effort restricted to the shorter survey window. The mask and `userdist` objects are the same as in the full-survey analysis so that the two analyses differ mainly in the data window.

```

output_file <- "tutorials/namkha_advanced/output/namkha_advanced_fitted_models_short.RData"

load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_capthist.RData")
load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_mask.RData")

# Check the shortened capture history and covariates before fitting models.
summary(short_ch)

```

```

Object class      capthist
Detector type     count
Detector number   55
Average spacing   2504.368 m
x-range           534489 576620 m
y-range           3316462 3360989 m

```

Usage range by occasion

```

      1
min    0
max  126

```

Counts by occasion

	1	Total
n	21	21
u	21	21
f	21	21
M(t+1)	21	21
losses	0	0
detections	30	30
detectors visited	17	17
detectors used	40	40

Individual covariates

```

      sex
Female : 5
Male   :13
Unknown: 3

```

```

names(covariates(short_traps))

```

```

[1] "Topography" "cliff"      "hill"      "valley_stream"
[5] "Water"      "Altitude"

```

```
names(covariates(mask_model))
```

```
[1] "tri"          "d2hydro"      "elev"         "std_tri"      "std_d2hydro"  
[6] "std_elev"
```

The candidate set mirrors the full-survey model set closely. This makes later comparisons easier to interpret.

```
# This analysis uses the shorter survey period, while still retaining the  
# elevation-constrained mask and the gorge-avoiding movement distances.
```

```
# Null model: density, encounter rate, and movement scale are constant.
```

```
s0 <- secr.fit(  
  short_ch,  
  detectfn = "HHN",  
  mask = mask_model,  
  model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1),  
  details = list(userdist = userd_model),  
  verify = FALSE,  
  trace = FALSE  
)
```

```
# Density varies with terrain ruggedness.
```

```
s1 <- secr.fit(  
  short_ch,  
  detectfn = "HHN",  
  mask = mask_model,  
  model = list(D ~ std_tri, lambda0 ~ 1, sigma ~ 1),  
  details = list(userdist = userd_model),  
  verify = FALSE,  
  start = s0,  
  trace = FALSE  
)
```

```
# Density varies with elevation.
```

```
s2 <- secr.fit(  
  short_ch,  
  detectfn = "HHN",  
  mask = mask_model,  
  model = list(D ~ std_elev, lambda0 ~ 1, sigma ~ 1),  
  details = list(userdist = userd_model),
```



```

    verify = FALSE,
    start = s0,
    trace = FALSE
  )

# Encounter rate varies by whether the camera is at a cliff.
s3 <- secr.fit(
  short_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ 1, lambda0 ~ cliff, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
  start = s0,
  trace = FALSE
)

# Combined model with ruggedness on density and local topography on encounter rate.
s4 <- secr.fit(
  short_ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ cliff + valley_stream, sigma ~ 1),
  details = list(userdist = userd_model),
  verify = FALSE,
  start = s1,
  trace = FALSE
)

AIC(s0, s1, s2, s3, s4)

```

				model		detectfn	npar
s4	D~std_tri	lambda0~cliff + valley_stream	sigma~1	hazard	halfnormal		6
s3		D~1	lambda0~cliff	sigma~1	hazard	halfnormal	4
s0		D~1	lambda0~1	sigma~1	hazard	halfnormal	3
s1		D~std_tri	lambda0~1	sigma~1	hazard	halfnormal	4
s2		D~std_elev	lambda0~1	sigma~1	hazard	halfnormal	4
	logLik	AIC	AICc	dAIC	AICwt		
s4	-61.88815	135.776	141.776	0.000	0.4578		
s3	-63.89672	135.793	138.293	0.017	0.4539		
s0	-67.14401	140.288	141.700	4.512	0.0480		
s1	-66.89073	141.781	144.281	6.005	0.0227		

```
s2 -67.14304 142.286 144.786 6.510 0.0177
```

```
save(
  s0, s1, s2, s3, s4,
  file = output_file
)
```

Multisession models

The multisession analysis uses `ch_ms`, `mask_ms`, and `userd_ms`. Each of these is a list aligned by session. The model parameters are shared across sessions unless a session term is included in the formula.

```
output_file <- "tutorials/namkha_advanced/output/namkha_advanced_fitted_models_ms.RData"

load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_capthist.RData")
load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_msmask.RData")

# Check the multisession capture history and covariates before fitting models.
summary(ch_ms)
```

```
$`1`
Object class      capthist
Detector type     count
Detector number   55
Average spacing   2504.368 m
x-range           534489 576620 m
y-range           3316462 3360989 m
```

```
Usage range by occasion
      1
min    0
max 133
```

```
Counts by occasion
      1 Total
n      20    20
u      20    20
f      20    20
M(t+1) 20    20
```

losses	0	0
detections	26	26
detectors visited	18	18
detectors used	46	46

Individual covariates

sex
 Female : 5
 Male :13
 Unknown: 2

\$`2`

Object class	capthist
Detector type	count
Detector number	55
Average spacing	2504.368 m
x-range	534489 576620 m
y-range	3316462 3360989 m

Usage range by occasion

1
 min 0
 max 117

Counts by occasion

	1	Total
n	16	16
u	16	16
f	16	16
M(t+1)	16	16
losses	0	0
detections	21	21
detectors visited	15	15
detectors used	37	37

Individual covariates

sex
 Female :4
 Male :9
 Unknown:3

```
names(covariates(traps_ms[[1]]))
```

```
[1] "Topography"      "cliff"           "hill"            "valley_stream"  
[5] "Water"           "Altitude"
```

```
names(covariates(mask_ms[[1]]))
```

```
[1] "tri"             "d2hydro"         "elev"            "std_tri"         "std_d2hydro"  
[6] "std_elev"
```

The first five models use the same broad structure as the full-survey models. The sixth model, `ms5`, includes `D ~ session` as an illustration of session-specific density, but this creates complications for extrapolation and is excluded from the results workflow.

```
ms0 <- secr.fit(  
  ch_ms,  
  detectfn = "HHN",  
  mask = mask_ms,  
  model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1),  
  details = list(userdist = userd_ms),  
  verify = FALSE,  
  trace = FALSE  
)  
  
ms1 <- secr.fit(  
  ch_ms,  
  detectfn = "HHN",  
  mask = mask_ms,  
  model = list(D ~ std_tri, lambda0 ~ 1, sigma ~ 1),  
  details = list(userdist = userd_ms),  
  verify = FALSE,  
  start = ms0,  
  trace = FALSE  
)  
  
ms2 <- secr.fit(  
  ch_ms,  
  detectfn = "HHN",  
  mask = mask_ms,  
  model = list(D ~ std_elev, lambda0 ~ 1, sigma ~ 1),
```

```

    details = list(userdist = userd_ms),
    verify = FALSE,
    start = ms0,
    trace = FALSE
  )

ms3 <- secr.fit(
  ch_ms,
  detectfn = "HHN",
  mask = mask_ms,
  model = list(D ~ 1, lambda0 ~ valley_stream, sigma ~ 1),
  details = list(userdist = userd_ms),
  verify = FALSE,
  start = ms0,
  trace = FALSE
)

ms4 <- secr.fit(
  ch_ms,
  detectfn = "HHN",
  mask = mask_ms,
  model = list(D ~ std_tri, lambda0 ~ cliff + valley_stream, sigma ~ 1),
  details = list(userdist = userd_ms),
  verify = FALSE,
  start = ms0,
  trace = FALSE
)

# Density varies by session, allowing different abundance in each time block.
# Note this model can't easily be used for extrapolation.
ms5 <- secr.fit(
  ch_ms,
  detectfn = "HHN",
  mask = mask_ms,
  model = list(D ~ session, lambda0 ~ 1, sigma ~ 1),
  details = list(userdist = userd_ms),
  verify = FALSE,
  start = ms0,
  trace = FALSE
)

AIC(ms0, ms1, ms2, ms3, ms4, ms5)

```

				model	detectfn	npar
ms4	D~std_tri	lambda0~cliff + valley_stream	sigma~1	hazard	halfnormal	6
ms3		D~1 lambda0~valley_stream	sigma~1	hazard	halfnormal	4
ms0		D~1 lambda0~1	sigma~1	hazard	halfnormal	3
ms5		D~session lambda0~1	sigma~1	hazard	halfnormal	4
ms2		D~std_elev lambda0~1	sigma~1	hazard	halfnormal	4
ms1		D~std_tri lambda0~1	sigma~1	hazard	halfnormal	4

	logLik	AIC	AICc	dAIC	AICwt
ms4	-101.3072	214.614	217.511	0.000	0.6796
ms3	-104.2658	216.532	217.822	1.918	0.2605
ms0	-107.4823	220.965	221.715	6.351	0.0284
ms5	-107.4703	222.941	224.231	8.327	0.0106
ms2	-107.4805	222.961	224.251	8.347	0.0105
ms1	-107.4818	222.964	224.254	8.350	0.0104

```
save(
  ms0, ms1, ms2, ms3, ms4, ms5,
  file = output_file
)
```

Sex-specific full-survey models

The final model-fitting script demonstrates two ways to include sex. Individual-covariate models use `hcov = "sex"` and can retain animals with unknown sex. Grouped models use `groups = "sex"` and require removing animals whose sex is unknown.

These models are saved to a separate file so they do not overwrite the main full-survey models.

```
output_file <- "tutorials/namkha_advanced/output/namkha_advanced_fitted_models_full_sex.RData"

load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_capthist.RData")
load("tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_mask.RData")

# Check the capture history and covariates before fitting sex-specific models.
summary(full_ch)
```

Object class	capthist
Detector type	count
Detector number	55
Average spacing	2504.368 m

```
x-range      534489 576620 m
y-range      3316462 3360989 m
```

Usage range by occasion

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
min	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
max	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	14	5

Counts by occasion

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
n	0	0	2	3	4	2	3	7	2	4	4	2	2	3	4	2	0	2	0	0
u	0	0	2	3	4	1	2	5	2	2	2	0	2	1	0	0	0	0	0	0
f	12	10	2	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
M(t+1)	0	0	2	5	9	10	12	17	19	21	23	23	25	26	26	26	26	26	26	26
losses	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
detections	0	0	2	3	4	2	3	7	3	4	4	2	2	3	4	2	0	2	0	0
detectors visited	0	0	2	3	4	2	2	7	3	4	4	2	2	3	4	2	0	2	0	0
detectors used	3	6	18	16	21	33	35	32	30	29	28	28	20	18	12	7	11	15	9	8
Total																				
n	46																			
u	26																			
f	26																			
M(t+1)	26																			
losses	0																			
detections	47																			
detectors visited	46																			
detectors used	379																			

Individual covariates

```
sex
Female : 8
Male   :15
Unknown: 3
```

```
names(covariates(full_traps))
```

```
[1] "Topography" "cliff"      "hill"      "valley_stream"
[5] "Water"      "Altitude"
```

```
names(covariates(mask_model))
```

```
[1] "tri"          "d2hydro"     "elev"        "std_tri"     "std_d2hydro"
```

```
[6] "std_elev"
```

```
# h2 models only use first two levels of the sex covariate
levels(covariates(full_ch)$sex)
```

```
[1] "Female" "Male" "Unknown"
```

In `hcov` models, `h2` represents the first two levels of the individual covariate supplied through `hcov = "sex"`. Here it is used on `lambda0` so that encounter rate can differ by sex while density remains a population-level surface.

```
# The fitting mask is the trap-buffer mask after clipping to plausible elevation.
# Custom user distances are used so movement can avoid the gorge barrier.
# ADVANCED ISSUE: hcov = "sex" allows detection parameters to vary by individual
# sex while still retaining animals with unknown sex.

# Null model with sex as an individual covariate on detection.
f0 <- secr.fit(full_ch, detectfn = "HHN", mask = mask_model,
  model = list(D ~ 1, lambda0 ~ h2, sigma ~ 1),
  hcov = "sex", details = list(userdist = userd_model), verify = FALSE, trace = FALSE)

# Density varies with terrain ruggedness.
f1 <- secr.fit(full_ch, detectfn = "HHN", mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ h2, sigma ~ 1),
  hcov = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0, trace = FALSE)

# Density varies with elevation.
f2 <- secr.fit(full_ch, detectfn = "HHN", mask = mask_model,
  model = list(D ~ std_elev, lambda0 ~ h2, sigma ~ 1),
  hcov = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0, trace = FALSE)

# Encounter rate varies by whether the camera is in a valley or stream setting.
f3 <- secr.fit(full_ch, detectfn = "HHN", mask = mask_model,
  model = list(D ~ 1, lambda0 ~ valley_stream + h2, sigma ~ 1),
  hcov = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0, trace = FALSE)

# Combined model with ruggedness on density and local topography on encounter rate.
f4 <- secr.fit(full_ch, detectfn = "HHN", mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ cliff + valley_stream + h2, sigma ~ 1),
  hcov = "sex", details = list(userdist = userd_model), verify = FALSE, start = f1, trace = FALSE)

AIC(f0, f1, f2, f3, f4)
```



```

                                model
f4 D~std_tri lambda0~cliff + valley_stream + h2 sigma~1 pmix~h2
f3          D~1 lambda0~valley_stream + h2 sigma~1 pmix~h2
f0          D~1 lambda0~h2 sigma~1 pmix~h2
f2          D~std_elev lambda0~h2 sigma~1 pmix~h2
f1          D~std_tri lambda0~h2 sigma~1 pmix~h2
      detectfn npar    logLik    AIC    AICc dAIC AICwt
f4 hazard halfnormal    8 -119.8368 255.674 264.144 0.000 0.5869
f3 hazard halfnormal    6 -122.3665 256.733 261.154 1.059 0.3456
f0 hazard halfnormal    5 -125.6102 261.220 264.220 5.546 0.0367
f2 hazard halfnormal    6 -125.4404 262.881 267.302 7.207 0.0160
f1 hazard halfnormal    6 -125.5164 263.033 267.454 7.359 0.0148

```

Grouped models estimate separate real parameters by sex. Because unknown-sex animals are removed, these models should not be compared in the same AIC table as the `hcov` models.

```

# ADVANCED ISSUE: groups = "sex" estimates separate real parameters by sex, but
# it requires removing individuals whose sex is unknown.
full_ch_known_sex <- subset(full_ch, subset = covariates(full_ch)$sex != "Unknown")
full_ch_known_sex <- shareFactorLevels(full_ch_known_sex)

summary(full_ch, terse = TRUE)

```

```

Occasions Detections    Animals Detectors
      20         47         26        55

```

```
summary(full_ch_known_sex, terse = TRUE)
```

```

Occasions Detections    Animals Detectors
      20         39         23        55

```

```

# Null grouped model for known-sex individuals.
f0g <- secr.fit(full_ch_known_sex, detectfn = "HHN", mask = mask_model,
  model = list(D ~ 1, lambda0 ~ g, sigma ~ 1),
  groups = "sex", details = list(userdist = userd_model), verify = FALSE, trace = FALSE)

# Grouped model with density varying with terrain ruggedness.
f1g <- secr.fit(full_ch_known_sex, detectfn = "HHN", mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ g, sigma ~ 1),
  groups = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0g, trace

```

```
# Grouped model with density varying with elevation.
f2g <- secr.fit(full_ch_known_sex, detectfn = "HHN", mask = mask_model,
  model = list(D ~ std_elev, lambda0 ~ g, sigma ~ 1),
  groups = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0g, trace

# Grouped model with encounter rate varying at valley/stream cameras.
f3g <- secr.fit(full_ch_known_sex, detectfn = "HHN", mask = mask_model,
  model = list(D ~ 1, lambda0 ~ valley_stream + g, sigma ~ 1),
  groups = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0g, trace

# Grouped combined model with density and encounter-rate covariates.
f4g <- secr.fit(full_ch_known_sex, detectfn = "HHN", mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ cliff + valley_stream + g, sigma ~ 1),
  groups = "sex", details = list(userdist = userd_model), verify = FALSE, start = f0g, trace

# Do not compare hcov and grouped models in the same AIC table because they use
# different data or parameterisations.
AIC(f0, f1, f2, f3, f4)
```

					model				
f4	D~std_tri	lambda0~cliff + valley_stream + h2	sigma~1	pmix~h2					
f3		D~1	lambda0~valley_stream + h2	sigma~1	pmix~h2				
f0			D~1	lambda0~h2	sigma~1	pmix~h2			
f2		D~std_elev	lambda0~h2	sigma~1	pmix~h2				
f1		D~std_tri	lambda0~h2	sigma~1	pmix~h2				
		detectfn	npar	logLik	AIC	AICc	dAIC	AICwt	
f4	hazard	halfnormal	8	-119.8368	255.674	264.144	0.000	0.5869	
f3	hazard	halfnormal	6	-122.3665	256.733	261.154	1.059	0.3456	
f0	hazard	halfnormal	5	-125.6102	261.220	264.220	5.546	0.0367	
f2	hazard	halfnormal	6	-125.4404	262.881	267.302	7.207	0.0160	
f1	hazard	halfnormal	6	-125.5164	263.033	267.454	7.359	0.0148	

```
AIC(f0g, f1g, f2g, f3g, f4g)
```

			model		detectfn	npar	
f3g	D~1	lambda0~valley_stream + g	sigma~1	hazard	halfnormal	5	
f4g	D~std_tri	lambda0~cliff + valley_stream + g	sigma~1	hazard	halfnormal	7	
f0g		D~1	lambda0~g	sigma~1	hazard	halfnormal	4
f1g		D~std_tri	lambda0~g	sigma~1	hazard	halfnormal	5
f2g		D~std_elev	lambda0~g	sigma~1	hazard	halfnormal	5

	logLik	AIC	AICc	dAIC	AICwt
f3g	-194.3884	398.777	402.306	0.000	0.6637
f4g	-193.1579	400.316	407.782	1.539	0.3074
f0g	-199.3268	406.654	408.876	7.877	0.0129
f1g	-198.7941	407.588	411.118	8.811	0.0081
f2g	-198.8246	407.649	411.179	8.872	0.0079

```
save(
  f0, f1, f2, f3, f4,
  f0g, f1g, f2g, f3g, f4g,
  file = output_file
)
```

What comes next

The next tutorial reports the full-survey, shortened-survey, and multisession results. It estimates abundance and density over the elevation-constrained Namkha survey area, makes predicted density maps, and plots density and detection covariate effects.